

Evolving Cryptographic Boolean Functions with Reaction Systems

Rocco Ascone
University of Trieste
Trieste, Italy
rocco.ascone@phd.units.it

Luca Manzoni
University of Trieste
Trieste, Italy
lmanzoni@units.it

Luca Mariot
University of Twente
Enschede, The Netherlands
l.mariot@utwente.nl

Gloria Pietropolli
University of Trieste
Trieste, Italy
gloria.pietropolli@units.it

Abstract

Designing bent Boolean functions for cryptographic applications is a challenging combinatorial task due to the super-exponential growth of the search space. We propose Evolutionary Boolean Reaction Systems (EvoBRS), an optimization method based on Reaction Systems (RS)—a bio-inspired model abstracting biochemical reactions. EvoBRS finds functions with competitive nonlinearity while providing a compact and interpretable representation. Unlike traditional methods such as Genetic Algorithms (GA), which rely on full truth tables, EvoBRS leverages a more expressive yet concise encoding.

CCS Concepts

• **Theory of computation** → **Cryptographic primitives; Evolutionary algorithms**; • **Computing methodologies** → *Bio-inspired approaches*; • **Security and privacy** → *Block and stream ciphers*.

Keywords

Reaction Systems, Evolutionary Algorithms, Evolutionary Reaction Systems, Bent functions, Cryptographic Boolean functions

ACM Reference Format:

Rocco Ascone, Luca Mariot, Luca Manzoni, and Gloria Pietropolli. 2025. Evolving Cryptographic Boolean Functions with Reaction Systems. In *Genetic and Evolutionary Computation Conference (GECCO '25 Companion)*, July 14–18, 2025, Malaga, Spain. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3712255.3726685>

1 Introduction

Designing Boolean functions with strong cryptographic properties is a challenging task, as they must satisfy multiple, often conflicting, criteria like nonlinearity, balancedness, and correlation immunity [20]. These functions play a crucial role in cryptography, serving as key components in block and stream ciphers to resist attacks

and ensure secure encryption. The search for such functions is a complex combinatorial optimization problem, with the number of candidates increasing super-exponentially as the input size grows. As a result, finding new ways to generate cryptographic Boolean functions remains an active research area, with applications spanning multiple domains [4, 21]. Boolean functions for cryptography are typically generated via algebraic constructions, random generation, or (meta-)heuristic methods. Algebraic approaches work for many different Boolean function sizes but lead to the same outputs for the same initial conditions, and designing new ones is non-trivial. Random generation is infeasible due to the vast search space. Among metaheuristics, Evolutionary Computation (EC) has proven effective in a variety of domains [13, 14, 18], with two main paradigms: Genetic Algorithms (GA), based on bitstring encodings, and Genetic Programming (GP), which uses tree-based symbolic representations. Both have been applied to the evolution of Boolean functions, showing promising results in cryptographic contexts. For a more detailed analysis, we refer the reader to [10], which comprehensive reviews of the principal EC paradigms proposed for generating Boolean functions.

A representation not yet explored in this context are Reaction Systems (RS). Introduced by Ehrenfeucht and Rozenberg [11, 12], they are a simple yet intuitive computational model inspired by biochemical reactions and have been successfully used to model diverse biological processes. The authors of [15, 16] introduced Evolutionary Reaction Systems (EvoRS), an evolutionary strategy to evolve RS, achieving interesting results among different real-world problems. EvoRS offers several advantages: it does not require the definition of a problem-specific set of functional or terminal symbols, and individuals can be directly read and understood by humans. Expanding on this idea, we propose Evolutionary Boolean Reaction Systems (EvoBRS), a novel evolutionary framework specifically designed to evolve individuals that represent Boolean functions in Disjunctive Normal Form (DNF).

A natural question arises: why to introduce a new representation for Boolean functions when many alternatives already exist? The main advantage is that EvoBRS individuals are compact and interpretable. In GA-based approaches, the truth table representation requires a binary vector of length 2^n (where n is the number of input variables), and converting it to a compact Boolean circuit could be computationally unfeasible. On the other hand, RS provide a more compact encoding where the circuit size can be forced to

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

GECCO '25 Companion, July 14–18, 2025, Malaga, Spain

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1464-1/2025/07

<https://doi.org/10.1145/3712255.3726685>

be polynomially bounded. Moreover, RS representations are more interpretable than solutions generated by GP, which often result in overly complex structures [7].

To validate our approach, we test it on the well-known problem of designing bent Boolean functions. We compare against a recent study [17] that evaluates several GA variants with improved operators. We do not compare our method with GP, as EvoRS is more similar to GA, and GA-based models have not yet matched the success rates reported for GP [10]. Our results show that EvoBRS can generate Boolean functions with strong cryptographic properties which, due to their RS-based representations are also inherently compact and interpretable.

2 Boolean Functions

This section covers the preliminary definitions and results related to Boolean functions used throughout the paper. The interested reader is referred to Carlet's book [6] for an in-depth discussion of Boolean functions and their properties relevant to cryptography.

Let $\mathbb{F}_2 = \{0, 1\}$ and, for any $n \in \mathbb{N}$, let $\mathbb{F}_2^n$ denote the set of all n -tuples of bits. A *Boolean function* of n variables is a mapping $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$. A natural way to uniquely represent such a function is by its *truth table*, which is the 2^n -bit vector Ω_f that specifies, for each possible input $x \in \mathbb{F}_2^n$ in lexicographic order, the corresponding output value $f(x)$. A critical property of Boolean functions in cryptography is their *nonlinearity*, which measures the Hamming distance of the function from the set of all linear functions. The nonlinearity $nl(f)$ of any Boolean function f with n inputs satisfies $nl(f) \leq 2^{n-1} - 2^{\frac{n}{2}-1}$, called the *covering radius bound*. In particular, the functions whose nonlinearity equals the maximal value $2^{n-1} - 2^{\frac{n}{2}-1}$ are called *bent*, and they can only exist for even values of n . Now, we formally define the combinatorial optimization problem investigated in this work:

Problem (Bent). Let $n \in \mathbb{N}$ be even. Find a n -variable Boolean function $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ such that f has maximum nonlinearity, i.e. f is bent.

Since the number of Boolean functions grows super-exponentially in n , it is unfeasible to perform an exhaustive search of all Boolean functions of 6 or more variables.

Disjunctive Normal Form. We can represent Boolean functions via logical formulas, which combine input variables through the logical operators AND, OR, and NOT. A *literal* refers to either an input variable or its negation, while a *clause* is a conjunction (AND) of literals, with its *size* defined as the number of literals it contains. DNF is a particular representation of a logical formula structured as a disjunction (OR) of clauses. When working with the DNF representation, we use the following notation: $\wedge$ (AND), $\vee$ (OR) and $\neg$ (NOT). Any Boolean function can be represented in DNF, unfortunately not in a unique way, i.e., different logical formulas can rise to the same Boolean functions. Nevertheless a logical formula in DNF can be easily converted in a Boolean circuit [9].

3 Reaction Systems

This section introduces the preliminary definitions related to RS and explains how they can be used to represent Boolean functions in DNF form, which is the focus of their use in this work. For a more

comprehensive introduction to RS, we refer the interested reader to the survey by Brijder et al. [5]. RS were introduced by Rozenberg and Ehrenfeucht [11, 12] in 2004 as a formal model inspired by chemical reactions and a great deal of research was carried out in recent years [1–3, 14]. The model represents chemical substances within a cell with a finite set of entities and the interactions among them as a finite set of rules called *reactions*. A *reaction* over a finite set S of *entities* is a triple (R_a, I_a, P_a) of subsets of S ; where R_a is the set of *reactants*, I_a the set of *inhibitors*, and P_a the nonempty set of *products*. The *size* of a reaction is defined as $|R_a \cup I_a|$. In this paper, as in the original definition [11], reactants and inhibitors are allowed to be empty sets. The set of all reactions over S is denoted by $\text{rac}(S)$. A *Reaction System* (RS) $\mathcal{A} = (S, A)$ consists of the finite set of entities S , called the *background set*, and a set $A \subseteq \text{rac}(S)$ of reactions over S .

Any subset of S is called *state* of the RS. A reaction a is *enabled* in a state T if $R_a \subseteq T$ (all reactants are presents) and $I_a \cap T = \emptyset$ (no inhibitors are present). The *result function* $\text{res}_a : 2^S \rightarrow 2^S$ of a reaction a is defined as:

$$\text{res}_a(T) := \begin{cases} P_a & \text{if } a \text{ is enabled in } T \\ \emptyset & \text{otherwise.} \end{cases}$$

The definition extends naturally to sets of reactions: given any $T \subseteq S$ and $A \subseteq \text{rac}(S)$, we define $\text{res}_A(T) := \bigcup_{a \in A} \text{res}_a(T)$. Consistently, the result function $\text{res}_{\mathcal{A}}$ of the whole RS $\mathcal{A} = (S, A)$ is defined to be equal to res_A , i.e. the result function of the whole set of reactions of the RS.

3.1 Boolean Reaction Systems

Using RS, we can easily represent Boolean functions in DNF form, as shown in the following example:

Example 3.1. Let $\mathcal{A} = (S, \{r_1, r_2, r_3\})$ the reaction system where $S = \{x_1, x_2, x_3, \text{True}\}$ and the reactions are

$$\begin{aligned} r_1 &= (\{x_1\}, \{x_2, x_3\}, \{\text{True}\}) \\ r_2 &= (\{x_2\}, \{x_1, x_3\}, \{\text{True}\}) \\ r_3 &= (\{x_2, x_3\}, \{x_1\}, \{\text{True}\}). \end{aligned}$$

We can associate a function $f : \mathbb{F}_2^3 \rightarrow \mathbb{F}_2$ to $\mathcal{A}$ in the following way. The input variables of f are represented by the symbols x_1, x_2, x_3 . In this way, any state $T \subseteq \{x_1, x_2, x_3\}$ encodes an input for f where $x_i = 1$ iff $x_i \in T$, e.g., the state $T = \{x_1, x_2\}$ corresponds to the assignment $x_1 = 1, x_2 = 1, x_3 = 0$. The symbol *True* represents that the output of the Boolean function is 1. Furthermore, the reaction $r_1 = (\{x_1\}, \{x_2, x_3\}, \{\text{True}\})$ is enabled by $T = \{x_1, x_2\}$ iff $C_1(x_1, x_2, x_3) = x_1 \wedge \neg x_2 \wedge \neg x_3$ is satisfied by the assignment $x_1 = 1, x_2 = 1, x_3 = 0$. Thus for each reaction r_i , we can build a clause C_i such that r_i is enabled by a state T iff C_i is satisfied by the assignment associated by T . We define $f : \mathbb{F}_2^3 \rightarrow \mathbb{F}_2$ such that f is 1 with input T iff $\text{True} \in \text{res}_{\mathcal{A}}(T)$. Since $\text{True} \in \text{res}_{\mathcal{A}}(T)$ iff one of the reactions is enabled, f can be written in DNF as $f(x) = C_1(x) \vee C_2(x) \vee C_3(x) = (x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (x_2 \wedge \neg x_1 \wedge \neg x_3) \vee (x_2 \wedge x_3 \wedge \neg x_1)$.

Let $n \in \mathbb{N}$, $X = \{x_1, \dots, x_n\}$ and $S = X \cup \{\text{True}\}$. If not specified, we always restrict to RS of the type $\mathcal{A} = (S, A)$ where for each $a \in A$, $a = (R_a, I_a, \{\text{True}\})$ with $R_a, I_a \subseteq X$. We denote this particular kind of RS as Boolean Reaction Systems (BRS). Extending

the construction of previous example, any BRS is associated to a Boolean functions in DNF where each clause is paired with a reaction of the same size. In each reaction, the variables in the reactants (resp., inhibitors) appears as positive (resp., negative) literals in the clause. If the function f represented by the BRS $\mathcal{A}$ is bent then $\mathcal{A}$ will be called a *bent* BRS.

4 Evolutionary Boolean Reaction Systems

EvoRS is a GA variant where individuals are RSs, designed to evolve programs within the RS formalism [15]. We chose EvoRS for its advantages over other evolutionary techniques. First, RS are interpretable—their reactions and interactions can be directly understood, unlike black-box models. Second, unlike GP, EvoRS requires no problem-specific functional or terminal sets. The only needed parameter is the number of symbols in the reactions, as computation emerges from their interactions, simplifying the setup. We propose a new variant of this paradigm—*Evolutionary Boolean Reaction Systems* (EvoBRS)—for representing and evolving Boolean functions via BRS. Our EvoBRS shares the structure of [15] but differs in operators and fitness.

To initialize the EvoBRS population, each BRS (individual) requires a background set and a set of reactions. For a problem of dimension n , the background is $S = \{x_1, \dots, x_n, \text{True}\}$, and each reaction a tuple $(R, I, \{\text{True}\})$. Reactions are initialized using init_size_min and init_size_max , which set bounds for the number of reactions allowed.

We modified the crossover proposed in [15], as it is too destructive to our purpose. Our new crossover, applied with probability p_c , exchanges reactions by size. Given two BRSs $\mathcal{A} = (S, A)$ and $\mathcal{B} = (S, B)$, let A_i and B_i be their reactions of size i . For each size i , it generates $\mathcal{A}_i = (S, (A \setminus A_i) \cup B_i)$ and $\mathcal{B}_i = (S, (B \setminus B_i) \cup A_i)$ —swapping all reactions of size i .

We define two types of mutations for BRS. The first, *renewal*, replaces a BRS with a completely new one generated as in the initialization process with probability p_{ren} . This introduces new genetic material to increase diversity and help explore the search space, especially in the early stages of evolution. The second, *random reaction change*, is less disruptive: with probability p_m , it replaces up to one-third of the reactions in $\mathcal{A} = (S, A)$ with new ones of the same size, chosen randomly.

To simplify individuals, we apply the *minimization* operator that removes reactions with no effect on dynamics—similar to “dead code” in source code. The minimization operator is always applied and transforms each BRS $\mathcal{A} = (S, A)$ with a new BRS $\mathcal{B} = (S, B)$, where $B \subseteq A$ is the set of all maximal reactions in A . This operator is computationally inexpensive as it relies solely on syntax, not fitness. Details are in [15].

To evolve bent BRSs, we aim to maximize nonlinearity, defining the fitness as $\text{fit}_{\text{bent}}(f) = \text{nl}(f)$, where $\text{nl}(f)$ represents the nonlinearity of the function f .

To sum up, the evolutionary cycle begins with the generation of the initial population. Selection follows, with elitism preserving top individuals. Renewal mutation is then applied, after which parents are selected via tournament for crossover. A random reaction change mutation is applied next. Minimization then removes redundant reactions, and syntactic duplicates (BRS with identical

reaction sets) are discarded. At the end of each generation, we check the stopping criteria—either reaching the maximum number of evaluations or achieving the nonlinearity thresholds. If not, the process repeats.

5 Experimental Methodology

This section outlines the baseline algorithms and experimental settings used to evaluate EvoBRS. To ensure full reproducibility, we provide our code publicly¹.

In [17], the authors comprehensively evaluate and compare different GA methods for the Bent problem. We compare EvoBRS with the top-performing GA algorithms from that study: OP, CP, and MoO—named after their crossover strategies (one-point, zero-length, and map-of-ones crossover). We adopt the same experimental framework, including the evolution cycle, evolutionary operators, fitness functions, and hyperparameters.

We consider three Bent problem instances with $n = 6, 8, 10$ variables. As explained in Section 5, initialization requires setting init_size_min and init_size_max , which we set to (15, 40, 80) and (25, 80, 160), respectively, based on the input size. These are the only parameters that vary with dimension. While init_size_max could reach 2^n (the number of possible DNF clauses), we select much smaller values to ensure compact representations. We used a population size of $n_{\text{pop}} = 100$ and a tournament size of $n_t = 4$. The stopping criterion was set to 200 000 fitness evaluations, matching the competing algorithms for fairness. Mutation rates were 0.2 for random reaction change and 0.1 for renewal; crossover was applied with a probability 0.8. Elitism preserved the top five individuals. Each problem instance was run 30 times and we used the Wilcoxon rank-sum test to assess significance, testing the null hypothesis of equal distributions.

6 Experimental Results

Figure 1 shows results for the Bent problem, with each row corresponding to a different instance. The left plots track the average nonlinearity of the best individuals over time, while the right plots show the final nonlinearity distribution across 30 runs. Boxplots include p-values from pairwise tests; bold values indicate that EvoBRS significantly outperforms the competitor ($\alpha = 0.05$). The analysis focuses on comparisons between our method and competing algorithms.

These plots confirm that EvoBRS achieves competitive results. For $n = 6$, EvoBRS consistently reaches a nonlinearity of 26, with some runs achieving 27 and even the maximum value of 28, outperforming the other variants. However, for MoO, while the average nonlinearity achieved by EvoBRS is higher, the final fitness distribution shows no statistical difference. For $n = 8$, our approach improves over OP and shows no statistical difference compared to CP. Conversely, for $n = 10$, EvoBRS results in a lower nonlinearity compared to the other GA algorithms. That being said, it is essential to point out that EvoBRS starts with a significantly lower initial nonlinearity compared to other methods (e.g., below 450 for $n = 10$, compared to over 466 for others). However, the steadily increasing curve demonstrates its capacity for improvement, while other methods remain more stable. These trends suggest that with

¹<https://github.com/roccoasc1/EvoRSbent>

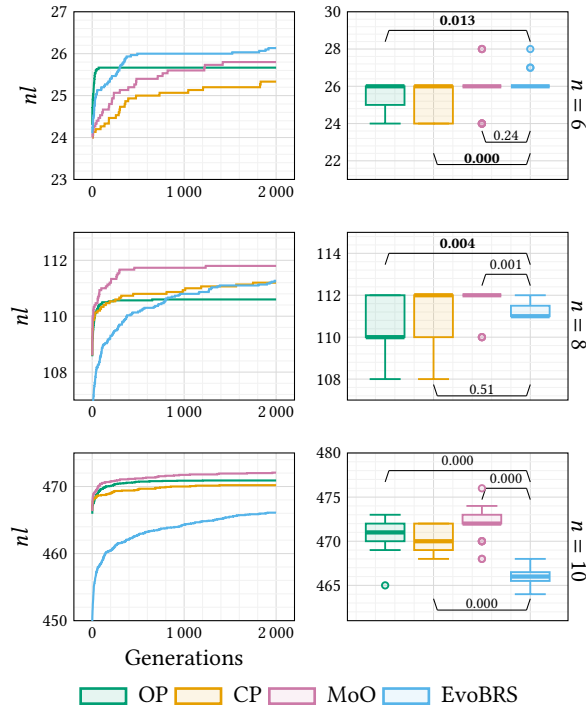


Figure 1: Average nonlinearity of the best individual per iteration (left) and final distributions across 30 runs (right) for the Bent problem with $n = 6, 8, 10$. Significant differences (Wilcoxon rank-sum) are annotated with p-values.

more fitness evaluations, EvoBRS could achieve even better performance. We suppose that the higher initial nonlinearity observed in the comparison GA algorithms is likely influenced by their initialization process. While EvoBRS employs random initialization, the comparison methods initialize populations with a desired level of unbalancedness, which is straightforward to implement with a GA representation but more challenging with a BRS representation and, more in general with compact representations. Interestingly, our method outperforms also a GP-based approach [8], despite using fewer fitness evaluations than in the referred study.

7 Conclusion and Future Directions

In this paper, we introduced EvoBRS, an evolutionary approach for evolving Boolean functions with strong cryptographic properties, based on the novel BRS representation derived from RS. We evaluated EvoBRS on a well-established cryptographic problem: generating bent functions. Compared to other GA-based algorithms, EvoBRS achieved competitive results. Unlike GP-based methods that often yield complex expressions, EvoBRS produces compact solutions directly in DNF and avoids the NP-hard minimization step required by standard GAs. This is the first attempt to evolve Boolean functions using RS, demonstrating the potential of compact representations. Future work should explore higher-dimensional cases, comparisons with GP-based methods, and improved initialization, as initial nonlinearity remains a limitation. Extending RS

to represent functions from $\mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ is another promising direction—potentially enabling novel S-box designs [19].

Acknowledgments

Cellular Automata Synthesis for Cryptography Applications (CASCA)" (P2022MPFRT) financed by the European Union – Next Generation EU.

References

- [1] Rocco Ascone, Giulia Bernardini, and Luca Manzoni. 2024. Fixed points and attractors of additive reaction systems. *Natural Computing* (2024), 1–11. <https://doi.org/10.1007/s11047-024-09977-2>
- [2] Rocco Ascone, Giulia Bernardini, and Luca Manzoni. 2024. Fixed points and attractors of reactantless and inhibitorless reaction systems. *Theoretical Computer Science* 984 (2024), 114322. <https://doi.org/10.1016/j.tcs.2023.114322>
- [3] Roberto Barbuti, Roberta Gori, Francesca Levi, and Paolo Milazzo. 2016. Investigating dynamic causalities in reaction systems. *Theor. Comput. Sci.* 623 (2016), 114–145. <https://doi.org/10.1016/j.tcs.2015.11.041>
- [4] Georg T Becker. 2015. The gap between promise and reality: On the insecurity of XOR arboriter PUFs. In *Cryptographic Hardware and Embedded Systems—CHES 2015: 17th International Workshop, Saint-Malo, France, September 13–16, 2015, Proceedings* 17. Springer, 535–555.
- [5] Robert Brijder, Andrzej Ehrenfeucht, Michael G. Main, and Grzegorz Rozenberg. 2011. A Tour of reaction Systems. *Int. J. Found. Comput. Sci.* 22, 7 (2011), 1499–1517.
- [6] Claude Carlet. 2021. *Boolean functions for cryptography and coding theory*. Cambridge University Press.
- [7] Claude Carlet, Marko Djurasevic, Domagoj Jakobovic, Luca Mariot, and Stjepan Picek. 2022. Evolving constructions for balanced, highly nonlinear boolean functions. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1147–1155.
- [8] Claude Carlet, Domagoj Jakobovic, and Stjepan Picek. 2021. Evolutionary algorithms-assisted construction of cryptographic boolean functions. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 565–573.
- [9] Yves Crama and Peter L. Hammer. 2011. *Boolean functions: Theory, algorithms, and applications*. Cambridge University Press.
- [10] Marko Djurasevic, Domagoj Jakobovic, Luca Mariot, and Stjepan Picek. 2023. A survey of metaheuristic algorithms for the design of cryptographic Boolean functions. *Cryptography and Communications* 15, 6 (2023), 1171–1197.
- [11] Andrzej Ehrenfeucht and Grzegorz Rozenberg. 2005. Basic notions of reaction systems. In *Developments in Language Theory: 8th International Conference, DLT 2004, Auckland, New Zealand, December 13–17, 2004. Proceedings* 8. Springer, 27–29.
- [12] Andrzej Ehrenfeucht and Grzegorz Rozenberg. 2007. Reaction systems. *Fundamenta informaticae* 75, 1–4 (2007), 263–280.
- [13] Steven Jorgensen, Giorgia Nadizar, Gloria Pietropoli, Luca Manzoni, Eric Medvet, Una-May O'Reilly, and Erik Hemmer. 2024. Large Language Model-based Test Case Generation for GP Agents. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 914–923.
- [14] Alberto Leporati, Luca Manzoni, Giancarlo Mauri, Gloria Pietropoli, and Claudio Zandron. 2023. Inferring P systems from their computing steps: An evolutionary approach. *Swarm and Evolutionary Computation* 76 (2023), 101223.
- [15] Luca Manzoni, Mauro Castelli, and Leonardo Vanneschi. 2012. Evolutionary reaction systems. In *European Conference on Evolutionary Computation, Machine Learning and Data Mining in Bioinformatics*. Springer, 13–25.
- [16] Luca Manzoni, Mauro Castelli, and Leonardo Vanneschi. 2013. A new genetic programming framework based on reaction systems. *Genetic Programming and Evolvable Machines* 14 (2013), 457–471.
- [17] Luca Manzoni, Luca Mariot, and Eva Tuba. 2020. Balanced crossover operators in genetic algorithms. *Swarm and Evolutionary Computation* 54 (2020), 100646.
- [18] Giorgia Nadizar and Gloria Pietropoli. 2023. A grammatical evolution approach to the automatic inference of P systems. *Journal of Membrane Computing* 5, 3 (2023), 129–143.
- [19] Kaisa Nyberg. 1991. Perfect nonlinear S-boxes. In *Workshop on the Theory and Application of Cryptographic Techniques*. Springer, 378–386.
- [20] Ryan O'Donnell. 2014. *Analysis of boolean functions*. Cambridge University Press.
- [21] Sayandeep Saha, Rajat Subhra Chakraborty, Srinivasa Shashank Nuthakki, Anshul, and Debdeep Mukhopadhyay. 2015. Improved test pattern generation for hardware trojan detection using genetic algorithm and boolean satisfiability. In *Cryptographic Hardware and Embedded Systems—CHES 2015: 17th International Workshop, Saint-Malo, France, September 13–16, 2015, Proceedings* 17. Springer, 577–596.